

3B2 FPGA Programming Lab Report

Liu Zihe^{1*}

¹University of Cambridge

Abstract

This report documents the design, implementation and testing of a traffic light controller on an Altera Cyclone V FPGA. A three-state finite state machine controlling a pedestrian crossing is described in Verilog, compiled, and configured onto a DE1-SoC board via JTAG, and is improved with a vehicle lockout.

1 Traffic Light Controller Design

We design a controller for a pedestrian light-controlled crossing, satisfying the following requirements:

1. The initial state is vehicle GREEN and pedestrian RED.
2. State changes occur only on demand: a pedestrian request or a reset.
3. After a request, the sequence is: vehicle YELLOW for 5 s, then vehicle RED with pedestrian GREEN for 10 s, then back to the initial state.
4. An asynchronous reset returns to the initial state from any state.

These requirements are captured by a three-state FSM shown in Figure 1. State G is the idle state with vehicle GREEN and pedestrian RED. A pedestrian request triggers a transition to state Y (vehicle YELLOW), which serves as a warning to vehicles. After a 5 s timeout the FSM enters state R (vehicle RED, pedestrian GREEN), allowing pedestrians to cross. A 10 s timeout then returns the FSM to state G. An asynchronous active-low reset forces the FSM back to state G from any state.

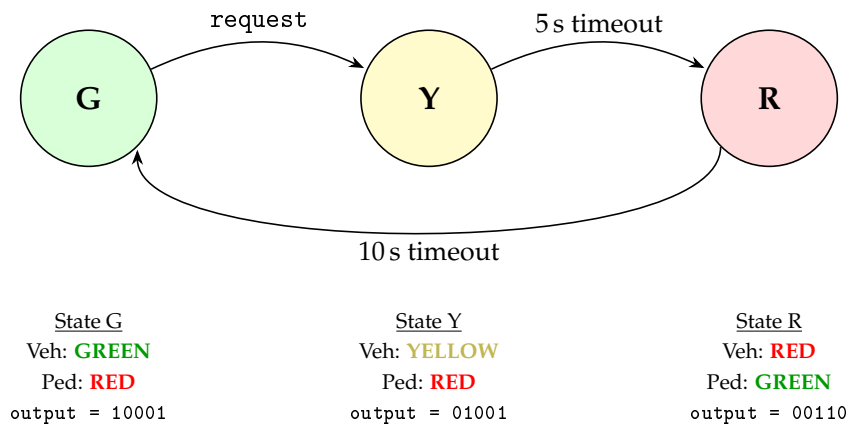


Figure 1: State diagram for the traffic light controller FSM. The 5-bit output vector is {veh_G, veh_Y, veh_R, ped_G, ped_R}. reset (active LOW, asynchronous) returns to state G from any state.

The outputs of each state and the transitions to the next states are summarised in Table 1. Each state activates exactly one vehicle light and one pedestrian light.

Table 1: State table for the traffic light controller.

State	Next State				Outputs				
	reset	request	5 s	10 s	veh_G	veh_Y	veh_R	ped_G	ped_R
G	G	Y	—	—	1	0	0	0	1
Y	G	—	R	—	0	1	0	0	1
R	G	—	—	G	0	0	1	1	0

*CRSID: zl559. Date: 20th February 2026

2 Verilog Implementation

Listing 1: Basic traffic light controller.

```
1 module tlc (
2     input wire      clk,
3     input wire      request,
4     input wire      reset,
5     output reg [4:0] \output // {veh_G, veh_Y, veh_R, ped_G, ped_R}
6 );
7
8     localparam G = 2'd0, Y = 2'd1, R = 2'd2;
9
10    reg [1:0] state;
11    reg [28:0] count;
12
13    // Sequential: state transitions & timer
14    always @(posedge clk or negedge reset) begin
15        if (!reset) begin
16            state <= G;
17            count <= 29'd0;
18        end else begin
19            case (state)
20                G: begin
21                    if (request == 1'b0) begin
22                        state <= Y;
23                        count <= 29'd0;
24                    end
25                end
26                Y: begin
27                    if (count == 29'd250_000_000) begin // 5s @ 50 MHz
28                        state <= R;
29                        count <= 29'd0;
30                    end else count <= count + 29'd1;
31                end
32                R: begin
33                    if (count == 29'd500_000_000) begin // 10s @ 50 MHz
34                        state <= G;
35                        count <= 29'd0;
36                    end else count <= count + 29'd1;
37                end
38                default: begin
39                    state <= G;
40                    count <= 29'd0;
41                end
42            endcase
43        end
44    end
45
46    // Combinational: output mapping
47    always @(*) begin
48        case (state)
49            G: \output = 5'b10001;
50            Y: \output = 5'b01001;
51            R: \output = 5'b00110;
52            default: \output = 5'b10001;
53        endcase
54    end
55
56 endmodule
```

The Verilog code in Listing 1 implements the FSM from Section 1. Key design choices made are:

- **Clock division.** The DE1-SoC provides a 50 MHz clock, but the TLC requires delays on the order of seconds. Rather than a separate clock divider module, a 29-bit counter (count) is used directly.
- **State encoding.** The three states are encoded with 2 bits (G=0, Y=1, R=2). A default case catches the unused encoding (3) and returns to state G.
- **Active-low inputs.** The DE1-SoC push-buttons output 0 when pressed. The reset is handled via `negedge reset` in the sensitivity list for asynchronous behaviour, and request is checked against `1'b0`.
- **Output mapping.** A combinational `always @(*)` block maps each state to a 5-bit output vector

[4:0] = {veh_G, veh_Y, veh_R, ped_G, ped_R}.

3 Compilation Results

The design was compiled in Quartus Prime 25.1 Lite Edition targeting the Cyclone V 5CSEMA5F31C6 device. The full Compiler Flow Summary screenshot is in Appendix A; key figures are in Table 2.

Table 2: Compiler Flow Summary.

Resource	Count
Logic utilisation (ALMs)	27
Total registers	32
Total pins	8

The 32 registers break down as follows: the 2-bit state register, the 29-bit count register, and 1 synchroniser flip-flop for the request input. The 27 Adaptive Logic Modules (ALMs) house the counter increment, ad state transition logic. The 8 pins match the 3 inputs (clk, request, reset) and 5 outputs (output[4:0]).

4 Pin Assignment

During initial compilation, Quartus is free to assign design signals to any FPGA pin. However, the DE1-SoC board has hardwired connections between specific FPGA pins and on-board components (LEDs, push-buttons, clock oscillator), so pin assignments must be set manually via the Assignment Editor to match. The assignments used are shown in Table 3. The design was recompiled after assigning pins so I/O is fitted correctly.

One limitation of the DE1-SoC is that all ten on-board user LEDs (LEDR0–LEDR9) are physically red. The traffic light colours (green, yellow, red) are therefore represented by LED position rather than colour: for example, LEDR8 represents vehicle GREEN even though the physical LED is red. Every output was placed at a different even integer position.

Table 3: Pin assignments for the DE1-SoC board.

Signal	Pin	Component
clk	PIN_AF14	50 MHz Clock
request	PIN_AA15	KEY1 (Push-button)
reset	PIN_AA14	KEY0 (Push-button)
output[0]	PIN_V16	LEDR0 (ped RED)
output[1]	PIN_V17	LEDR2 (ped GREEN)
output[2]	PIN_W17	LEDR4 (veh RED)
output[3]	PIN_Y19	LEDR6 (veh YELLOW)
output[4]	PIN_W21	LEDR8 (veh GREEN)

5 Testing

Table 4: Board testing results.

Initial State	Input Applied	Expected Next State	Pass/Fail
G	request	Y (5 s timer)	TODO
Y	5 s timeout	R (10 s timer)	TODO
R	10 s timeout	G	TODO
Any	reset	G	TODO

6 Improved Circuit

6.1 Updated State Machine

6.2 Verilog Implementation

6.3 Testing

7 Conclusions

A Appendix

A.1 Compilation Report

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Feb 20 13:21:29 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	tlc
Top-level Entity Name	tlc
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	27 / 32,070 (< 1 %)
Total registers	32
Total pins	8 / 457 (2 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 2: Quartus Prime Compiler Flow Summary for the basic TLC design.

A.2 Testbench

Listing 2: Testbench for the basic TLC.

```
1 // Testbench for basic Traffic Light Controller (tlc.v)
2 // Uses force/release on internal counter to skip long timer waits.
3 `timescale 1ns / 1ps
4
5 module tlc_tb;
6
7     reg clk, request, reset;
8     wire [4:0] out;
9
10    tlc uut (.clk(clk), .request(request), .reset(reset), .\output (out));
11
12    initial clk = 0;
13    always #10 clk = ~clk; // 50 MHz
14
15    localparam G = 2'd0, Y = 2'd1, R = 2'd2;
16    localparam OUT_G = 5'b10001, OUT_Y = 5'b01001, OUT_R = 5'b00110;
17    localparam Y_THRESH = 29'd250_000_000, R_THRESH = 29'd500_000_000;
18
19    integer pass_count = 0, fail_count = 0;
20
21    task check(input [63:0] name, input cond);
22        begin
23            if (cond) begin $display("PASS: %0s", name); pass_count = pass_count + 1; end
24            else begin $display("FAIL: %0s", name); fail_count = fail_count + 1; end
25        end
26    endtask
27
28    task wait_clks(input integer n);
29        integer i;
```

```

30     begin for (i = 0; i < n; i = i + 1) @(posedge clk); end
31 endtask
32
33 // Skip counter to near threshold, then let it trigger naturally
34 task skip_counter(input [28:0] thresh);
35     begin
36         force uut.count = thresh - 2;
37         wait_clks(1);
38         release uut.count;
39         wait_clks(5);
40     end
41 endtask
42
43 initial begin
44     $dumpfile("tlc_tb.vcd");
45     $dumpvars(0, tlc_tb);
46     request = 1; reset = 1;
47
48     // T1: Reset -> state G
49     $display("\n---T1: Reset---");
50     reset = 0; wait_clks(3);
51     check("state=G", uut.state == G);
52     check("out=10001", out == OUT_G);
53     reset = 1; wait_clks(2);
54
55     // T2: Full cycle G -> Y -> R -> G
56     $display("\n---T2: Full cycle---");
57     request = 0; wait_clks(2); request = 1;
58     check("G->Y", uut.state == Y);
59     check("out=01001", out == OUT_Y);
60     skip_counter(Y_THRESH);
61     check("Y->R", uut.state == R);
62     check("out=00110", out == OUT_R);
63     skip_counter(R_THRESH);
64     check("R->G", uut.state == G);
65     check("out=10001", out == OUT_G);
66
67     // T3: Async reset from Y and R
68     $display("\n---T3: Async reset---");
69     request = 0; wait_clks(2); request = 1;
70     check("in_Y", uut.state == Y);
71     reset = 0; wait_clks(2);
72     check("Y_reset->G", uut.state == G);
73     reset = 1; wait_clks(2);
74     // Now reset from R
75     request = 0; wait_clks(2); request = 1; wait_clks(1);
76     skip_counter(Y_THRESH);
77     check("in_R", uut.state == R);
78     reset = 0; wait_clks(2);
79     check("R_reset->G", uut.state == G);
80     reset = 1; wait_clks(2);
81
82     // T4: Idle stability
83     $display("\n---T4: Idle---");
84     request = 1; wait_clks(100);
85     check("stays_G", uut.state == G);
86
87     // Summary
88     $display("\n==%0d passed, %0d failed==", pass_count, fail_count);
89     if (fail_count == 0) $display("ALL TESTS PASSED");
90     else $display("SOME TESTS FAILED");
91     $finish;
92 end
93
94 endmodule

```